Last-Mile Delivery Tracker

System Architecture & Engineering Write-Up

-Gowri Ajith

1. System Overview & Architecture

The Last-Mile Delivery Tracker is a full-stack logistics platform built to manage last-mile delivery operations, rate card billing, automated agent assignment, immutable order tracking, notifications, and administrative analytics. Built with **Python FastAPI**, **SQLAlchemy ORM** (SQLite/PostgreSQL), and **React (Vite)**, the architecture enforces strict **Role-Based Access Control (RBAC)** across three personas: Admin, Delivery Agent, and Customer.

2. Core Operational Engines

2.1 Dynamic Rate Calculation Engine & Volumetric Billing

Delivery charges are computed dynamically by **app/services/rate_engine.py** without hardcoded pricing rules:

1. Volumetric Weight Formula

Volumetric Weight (kg) = (Length (cm) × Breadth (cm) × Height (cm)) / 5000

2. Billing Weight

Billing Weight = max (Actual Weight, Volumetric Weight)

3. Zone Traversal & Surcharge Lookup

- Intra-Zone: When origin pickup and destination drop pin-codes reside in same zone, base intra-zone rates per kg apply (₹40/kg B2C, ₹50/kg B2B).
- Inter-Zone: Cross-zone deliveries apply inter-zone rate cards (₹80/kg B2C, ₹100/kg B2B).
- COD Surcharge: Fixed fees (₹20 B2C, ₹30 B2B) are appended for Cash-on-Delivery orders.

2.2 Zone Detection & Area Mapping

Geographic pricing and routing rely on two normalized database models: Zone and Area.

- Administrators define geographic Zones (e.g., Central Zone, South Zone).
- Individual pincodes and neighborhoods are assigned to specific zones as Areas (e.g., Area pincode 560001 → Central Zone).
- Upon order placement, the system performs zone detection by looking up origin and destination pincodes against Area.zone_id. If pincodes fall into different zone IDs, the order is categorized as Inter-Zone; otherwise, Intra-Zone.

2.3 Intelligent Agent Auto-Assignment Logic

To minimize transit time and balance fleet workload, app/services/assignment_engine.py executes a proximity search using Euclidean spatial distance:

$$\text{Distance} = \sqrt{(\text{Lat}_{agent} - \text{Lat}_{pickup})^2 + (\text{Lng}_{agent} - \text{Lng}_{pickup})^2}$$

Search Workflow:

4. Query active (is_active = True) and available (is_available = True) agents.
5. Home Zone Prioritization: Filter agents whose registered home zone matches the pickup area's zone ID.
6. If multiple home-zone agents qualify, select the agent closest to the pickup origin coordinates.
7. City-Wide Fallback: If no available agent is located in the pickup zone, expand search city-wide to assign the nearest available agent.
8. If no agents are available, the order remains unassigned until an agent becomes free or an Admin manually assigns one.

2.4 Order Lifecycle State Machine & Failed Delivery Handling

Package state transitions follow a strict state machine to maintain audit integrity:

[CREATED] → [AGENT_ASSIGNED] → [PICKED_UP] → [IN_TRANSIT] → [OUT_FOR_DELIVERY] → [DELIVERED] / [FAILED]

Each status change generates an immutable record in OrderTrackingHistory recording actor ID, role, timestamp, and notes.

Failed Delivery Protocol

1. When a delivery attempt fails (customer absent or address issue), the agent flags the order status as FAILED with failure reasons.
2. An automated notification (SMS & Email) alerts the customer of the failed attempt.
3. The customer accesses the Rescheduling Portal to select a new delivery date and window.
4. Rescheduling shifts status to RESCHEDULED, re-queue the order, and re-evaluates agent assignment.
5. Strict policy limits rescheduling to 3 attempts maximum, after which Admin intervention is required.

3. Database Entity-Relationship Model

The relational schema comprises normalized tables:

- **users**: Credentials and roles (ADMIN, DELIVERY_AGENT, CUSTOMER).
- **zones & areas**: Zone boundaries and pincode mappings.
- **rate_cards**: Dynamic pricing matrices and volumetric factor settings.
- **agent_locations**: Live agent coordinates and availability flags.
- **orders**: Dimensions, calculated rates, current status, customer/agent links.
- **order_tracking_history**: Immutable append-only audit trail.
- **notifications**: Append-only log of SMS and email dispatches.

4. Verification & Quality Assurance

The codebase includes a comprehensive pytest automated test suite validating authentication, volumetric rate formulas, zone detection, auto-assignment fallback, lifecycle state machine constraints, 3-attempt rescheduling limits, notification logging, and admin KPI calculations.